

Offline Satellite Tile Classification Service

Design Note — Part 1

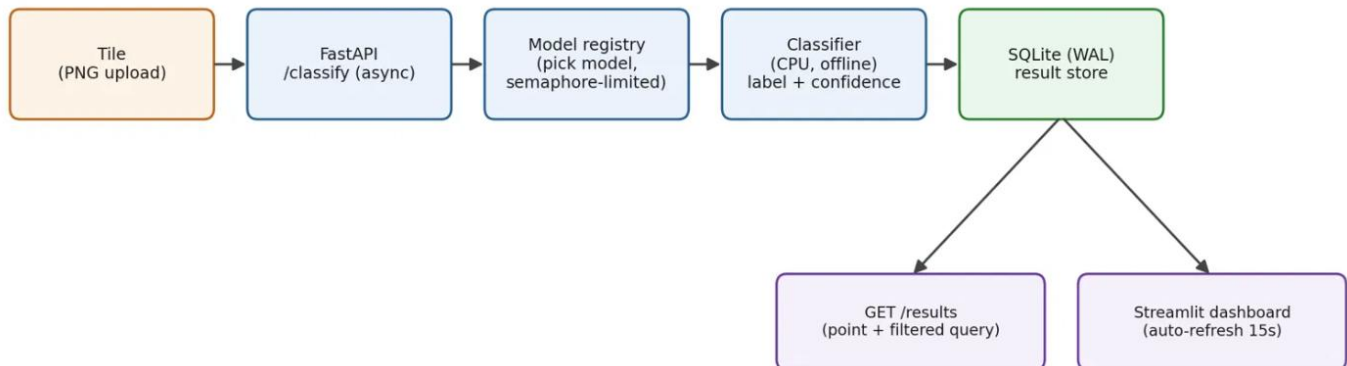
1. Goal and approach

Build an offline service that accepts a satellite tile, predicts one of seven land-use classes, stores the prediction with enough context to inspect it later, and gives an analyst practical ways to find results. The supplied tiles are small RGB images; the available classes are AnnualCrop, Forest, Highway, Industrial, Residential, River, and SeaLake. The design assumes a single host and modest volumes, so it favors a small local deployment over a distributed

The project has four main pieces:

1. Model and preprocessing: Training and evaluation scripts share the class mapping and image transforms. The service loads checkpoint files once at startup and offers both SmallCNN and fine-tuned ResNet18. The selected transform resizes and normalizes each image to match its model.
2. Classification API: FastAPI accepts one image per request, decodes it, runs CPU inference, and returns the predicted class, top probability, and full class-probability distribution. Inference runs in a worker thread with a concurrency limit so CPU work does not block the event loop.
3. Local persistence: The original tile is stored as a PNG on disk. SQLite stores its identifier, source filename, predicted class, confidence, all class probabilities, model name, review flag, and timestamp. WAL mode supports dashboard reads alongside API writes.
4. Analyst access: REST endpoints provide point lookup and paginated filtering by predicted class, confidence, model, and review flag. A Streamlit dashboard shows summary counts, label distribution, filters, and tile previews.

Workflow:



There is no queue or batch-ingest layer in the current version; each upload is handled as a request/response operation.

2. Decisions and trade-offs

- Model choice: SmallCNN is lightweight and can be trained without downloading pretrained weights. ResNet18 currently performs better on the supplied evaluation set, but it is heavier and its training requires a one-time ImageNet-weight download. The service currently keeps both available and chooses the checkpoint with the higher recorded validation accuracy by default, while allowing explicit selection. This is useful for comparison, but validation accuracy alone does not settle which model is

fit for a real analyst workflow. I would choose the production default only after agreeing on the acceptable error profile, compute budget, and whether setup must be fully air-gapped.

- Uncertain predictions: The service stores every prediction and marks results below a 0.6 top-softmax-probability threshold for review. Keeping the full distribution lets analysts inspect near-ties and apply different filters without rerunning inference. The threshold is a triage rule, not a guarantee: neural-network softmax scores are not necessarily calibrated, and a confidently wrong result may not be flagged. Before using confidence to automate decisions, I would measure calibration and class-specific error rates on representative, labeled data. If a class has a costly failure mode, its review policy may need to be stricter than a single global cutoff.
- What to store and how to identify a tile: Keeping image bytes as files and queryable metadata in SQLite avoids putting image blobs in the database while retaining previews and traceability. The current tile ID combines the source filename stem with a truncated content hash, and SQLite enforces uniqueness on that ID. Re-uploading the same file therefore updates its existing row; it does not create a prediction-history record. That is compact and convenient for a latest-result view, but it is not an audit trail. If analysts need to compare model versions or preserve human corrections, I would separate tile identity from prediction records and append a new prediction for each model/run, optionally linking reviewed labels to the original prediction.
- What querying means: The core needs are point lookup (“what happened to this tile?”), filtered browsing (“show low-confidence Rivers from this model”), and quick monitoring (“are labels or review counts changing?”). The API supports the first two with filters and pagination; the dashboard adds basic aggregate views and thumbnails. The dashboard reads the same SQLite database directly, which keeps this small deployment simple, but means both services must share the same database and image-storage paths. If they are deployed on separate hosts or the data becomes large, I would put the dashboard behind API query endpoints or move to a shared database rather than rely on a shared local file.

3. Assumptions and open questions

- “Offline” means no hosted inference calls at runtime. Must training and installation also work without internet access? That determines whether the pretrained ResNet option is acceptable.
- The current system is a demo without authentication, authorization, or restricted CORS. Who may submit tiles and see results in the intended deployment?
- What throughput, latency, and retention are expected? A sustained burst could call for a durable queue; long retention or larger volumes could change the storage choice.
- Retention policy — keep every tile/prediction forever, or age out old ones?
- Do corrected predictions (analyst overrides) feed back into retraining and do we retrain the model and at what gap we retrain?

NOTE:

For the current assignment, the simplest defensible choice is to keep the local FastAPI + SQLite + file-store + Streamlit design, expose uncertainty rather than discard it, and be explicit that review flags and model confidence are aids for triage—not correctness guarantees.